

## DMP 02, 03 & 04 Lecture Summary

### Overview

- Python is an object-oriented language and classes form the basis for all data types.
- Object-oriented Programming, or OOP for short, is a programming paradigm that provides a means of structuring programs so that properties and behaviours are bundled into individual objects.
- Using OOP helps us to write code efficiently without repetition as we can leverage existing code. In addition, it also helps us to manage the complexity through a modularized approach.
- We can utilize OOP to create backtester classes, which can help us create multiple backtests efficiently.
- The two types of backtesting are vectorized backtesting and events-driven backtesting. Both can be implemented using the OOP approach.

### Basic Concepts in OOP

#### Class:

The class in OOP is the blueprint which can be used to create many instances. Class contains methods and attributes that can be shared by all potential instances.

For example, Dog is a specific species or a 'class' and most animals from this class are expected to have attributes such as 4 legs, bark, be human-friendly etc..

#### **Examples of some built-in classes in Python:**

- *int* class for integers
- *float* class for floating-point values
- *str* class for character strings

Here is a sample code snippet that shows the syntax for creating a custom user-defined class in Python. We create a class called 'student' using the class keyword (in green) and create a variable inside it called 'goal'.

```
class student:  
    goal = 'get educated'
```

#### Object:

An object is a specific instance of a class, which can have its own attributes and functionalities.

For example, Bruno, Fluffy, Scooby are all instances of the Dog class with their own unique attributes such as breed, average height, jaw structure, tricks they can perform etc.

Below, we show the syntax for creating an object of class 'student' created earlier:

```
# Instantiation
John = student()
```

Thus, the object 'John' is one specific student or in other words, one specific instance of the class 'student'.

The `isinstance()` function is used to determine if an instance belongs to a certain class.

For the above example, it returns a value of True, as the object 'John' is an instance of the class 'student'.

```
# function to check if an object belongs to a class
isinstance(John, student)
```

True

---

### Class attributes and methods:

**Attributes** are the features/properties of the objects or the variables used in a class.

**Methods** are the operations or activities performed by that object. These are defined as functions in the class.

**Class variables/attributes and class-methods** belong to the class itself and do not vary based on the instances. For example, in the student class, 'goal' is a class attribute and 'print\_goal' is a class method.

While creating a class method we use the '@classmethod' decorator before defining the body of the function. This is a special way of telling Python that we are creating a method linked to the class:

```
class student:
    goal = 'get educated' # a class attribute

    @classmethod
    def print_goal(cls):    # a class method
        print(cls.goal)
        print('learn, earn and enjoy!')
```

### Instance attributes and methods:

**Instance variables/attributes**, on the other hand, are owned by the instance/object of the class, so they vary depending on the details of the specific objects. They are initialized inside the **\_\_init\_\_() method**, which is a special function that is invoked each time an object is created. It is commonly referred to as the constructor method of the class.

Note that the **'self'** keyword identifies the instance upon which a method is invoked. All the instance attributes are linked to this self keyword as it links them to the specific object represented by the self.

**Instance methods** can access unique data/variables of an instance/object through the **'self'** keyword as **'self'** is passed as the first argument in any instance method. Instance methods are the most common type of methods you will find in a class.

Below, in the highlighted section, we initialize two instance attributes and define an instance method **'total\_score'**:

```
class student:
    goal = 'get educated' # a class attribute

    @classmethod
    def print_goal(cls): # a class method
        print(cls.goal)
        print('learn, earn and enjoy!')

    # initialization function required to construct different objects/instances
    def __init__(self, science_marks, arts_marks):
        self.science = science_marks # instance attribute 1
        self.arts = arts_marks # instance attribute 2

    def total_score(self): # instance method
        print('Total score is:', self.science+self.arts )
```

### Static methods:

Finally, there are **static methods** that can be created inside a class. Static methods are just normal functions under the scope of a class. Hence, they don't pass **'self'** or **'cls'** as the first argument as static methods do not depend on or have access to any instance or class data.

Below, in the highlighted section, create a static method using the **@staticmethod** decorator:

```
class student:
    goal = 'get educated' # a class attribute

    @classmethod
    def print_goal(cls): # a class method
        print(cls.goal)
        print('learn, earn and enjoy!')

    # initialization function required to construct objects/instances
    def __init__(self, science_marks, arts_marks):
        self.science = science_marks # instance attribute 1
        self.arts = arts_marks # instance attribute 2

    def total_score(self): # an instance method
        print('Total score is:', self.science + self.arts)

    @staticmethod
    def print_current_year(year): # a static method
        print('The current year is:', year)
```

### Inheritance:

In the world of OOP, a child class(sub class) is said to be inherited from its parent class(super class) when it inherits all the attributes and methods from the parent class. In addition, one could always add, remove or override the attributes and methods inherited from the parent class.

Below, we show the syntax for creating a child class:

```
: #syntax
class name_of_the_child(name_of_the_parent):
    pass
```

For example, we can create a new subclass/child class called 'geeky student' from the parent class 'student'(shown earlier) and override the 'goal' variable for the 'geeky\_student' class as follows:

```
# The 'geeky_student' class inherits all the methods and attributes from the 'student' class
# In addition we override the 'goal' attribute
class geeky_student(student):
    goal = 'Study like crazy!'
```

The `issubclass()` function is used to determine if a class is a child class of another class.

For the above example, it returns a value of True, as the class 'geeky\_student' is a subclass of the class 'student':

```
issubclass(geeky_student, student)
```

True

---

Note that a child class can have multiple parent classes as well.

### **Benefits of using OOP approach for Backtesting**

Once we have coded the strategy, the next step is to simulate it on historical data to test how well it would have performed in the past. This process is called backtesting the strategy.

Using OOP, we can create a 'backtester class' which helps to get the data and functions required under one systematic structure. That is, it helps to adopt a modular approach by creating functions for different tasks that need to be performed in a backtest such as fetching the data, creating the indicators, generating signals etc. within a single umbrella of a class.

Each new backtest then is just a specific instance of the backtester class.

In addition, utilizing OOP to create backtests has the following advantages:

- As classes are highly flexible and extensible, we can easily modify or build upon existing classes later to extend our operations using the concept of inheritance.
- Leveraging existing code saves time and energy and different instances of the backtester class help us to analyse backtests with different parameters and tickers easily.

### **Vectorized Backtesting vs Event-Driven Backtesting**

There are two popular approaches to build a backtest namely, the vectorized approach and the event-driven approach.

#### **Vectorised Backtesting:**

This is especially useful in the explorative stage of strategy research. In this approach, all the data related to a strategy is organized into vectors (matrices) and then passed to the strategy code as input to simulate the performance of the strategy on the historical data.

Pros: Because this approach leverages the strength of linear algebra via powerful python packages like pandas and numpy, it is drastically faster than event-driven backtesting.

Cons: Because all the data (from different time points) is passed in as a vector, there is always a danger of introducing look-ahead bias. In addition, a vectorized approach is not always feasible for testing more complex strategies.

Below, we show an OOP implementation of a vectorized backtest for a simple moving average crossover strategy (as covered in the DMP 02 lecture). Notice that we have used vectorization through the use of `np.where()` function to generate the signals.

```
class SMA_vectorized_backtester:

    def __init__(self, ticker, start_date, end_date, ma_short, ma_long):
        self.ticker = ticker
        self.start_date = start_date
        self.end_date = end_date
        self.ma_short = ma_short
        self.ma_long = ma_long

        #Call the basic methods in the __init__ constructor itself so that they are automatically executed
        self.fetch_data()
        self.indicators()
        self.signals()
        self.positions()
        self.returns()

    def fetch_data(self):
        self.df = yf.download(self.ticker, self.start_date, self.end_date)

    def indicators(self):
        self.df['ma_short'] = self.df['Adj Close'].rolling(window=self.ma_short, center=False).mean()
        self.df['ma_long'] = self.df['Adj Close'].rolling(window=self.ma_long, center=False).mean()
        self.df['ma_short_prev'] = self.df['ma_short'].shift()
        self.df['ma_long_prev'] = self.df['ma_long'].shift()
        self.df.dropna(inplace=True)

    def signals(self):
        self.df['signal'] = np.where((self.df['ma_short'] > self.df['ma_long'])
                                     & (self.df['ma_short_prev'] < self.df['ma_long_prev']), 1, 0)

        self.df['signal'] = np.where((self.df['ma_short'] < self.df['ma_long'])
                                     & (self.df['ma_short_prev'] > self.df['ma_long_prev']), -1, self.df['signal'])

    def positions(self):
        self.df['position'] = self.df['signal'].replace(to_replace=0, method='ffill')

    def returns(self):
        self.df['bnh_returns'] = np.log(self.df['Adj Close'] / self.df['Adj Close'].shift(1))
        self.df['strategy_returns'] = self.df['bnh_returns'] * self.df['position'].shift(1)
        print('Total return:', self.df['strategy_returns'].cumsum()[-1])
        return self.df['strategy_returns'].cumsum()[-1]
```

### **Event-driven backtesting:**

In this approach, we pass one event/bar after the other to the strategy in a sequential manner instead of passing a vector (all bars) at once. The core of the event-driven approach is a loop that iterates over time, simulating the prices and positions taken/ orders placed depending on the market signals.

Pros: This approach is much closer to reality as in real world trading we don't have the full scale of data available at once, but rather we get the data sequentially at random intervals. Passing data in sequential manner ensures that our strategy is not making use of any information that is not available up until that point in time and this helps avoid

the danger of look-ahead bias. In addition, this approach is very flexible, as complex conditions can be introduced and tested in a strategy easily.

Cons: Event-driven backtest is not as fast as a vectorized backtest as it makes use of a loop.

The following snippet shows an example of an event-driven backtest (notice the the for-loop iterating over bars) implemented using the OOP approach:

```
class SMAEventBacktester(BacktestingBase):
    def _prepare_statistics(self):
        self.data['SMA1'] = self.data[self.symbol].rolling(self.SMA1).mean()
        self.data['SMA2'] = self.data[self.symbol].rolling(self.SMA2).mean()
        self.data['signal'] = np.where(self.data['SMA1'] > self.data['SMA2'], 1, -1)
    def backtest_strategy(self, SMA1, SMA2):
        self.SMA1 = SMA1
        self.SMA2 = SMA2
        self.trades = 0
        self.position = 0
        self.units = 0
        self.current_balance = self.initial_balance
        self._prepare_statistics()
        print(f'*** STARTING BACKTEST ***')
        print(f'SMA1 = {SMA1} | SMA2 = {SMA2}')
        for bar in range(self.SMA2 - 1, len(self.data)):
            date, price = self.get_date_price(bar)
            if self.position in [0, -1] and self.data['signal'].iloc[bar] == 1:
                # enter long position
                print(55 * '=')
                print(f'{date} | *** GOING LONG ***')
                self.place_buy_order(bar, units=(1 - self.position) * 5000)
                self.position = 1
                self.print_current_balance(bar)
                self.print_net_wealth(bar)
            elif self.position in [0, 1] and self.data['signal'].iloc[bar] == -1:
                # enter short position
                print(55 * '=')
                print(f'{date} | *** GOING SHORT ***')
                self.place_sell_order(bar, units=(1 + self.position) * 5000)
                self.position = -1
                self.print_current_balance(bar)
                self.print_net_wealth(bar)
        self.close_out(bar)
```